

Universidad de La Sabana

**INFORME DE ANÁLISIS Y DISEÑO
ARQUITECTÓNICO DE SOFTWARE**

Grupo: 6503

Integrantes:

Pallares Kelin, Daniel Felipe - 322826

Juan Camilo Gomez Bayona - 0000323015

Jeronimo Infante Vega - 0000324197

Daniel Sanchez Sotelo - 0000325069

Samuel Guerrero Arcos - 0000323500

Profesor: Wilmer Fabian Triana Pulgarin

Diseño y Arquitectura de Software

2026-1

Contenido

- 1. Introducción
- 2. Descripción del problema
- 3. Objetivo del proyecto
- 4. Alcance del sistema
- 5. Actores y módulos principales
- 6. Requerimientos funcionales y reglas de negocio
- 7. Diagrama de clases UML
- 8. Evaluación de patrones de diseño aplicables
- 9. Atributos de calidad
- 10. Estilo arquitectónico y evolución
- 11. Stack tecnológico propuesto
- 12. Repositorio y estructura del proyecto
- 13. Conclusiones
- 14. Referencias

1. Introducción

FinWise AI es una propuesta de sistema de información orientada a estudiantes universitarios y jóvenes adultos que necesitan administrar mejor sus ingresos, gastos, presupuestos y metas de ahorro. La idea nace de una situación común: muchas personas comienzan su vida financiera sin herramientas claras para entender en qué gastan, cuánto pueden ahorrar y qué decisiones deberían estudiar antes de invertir.

El proyecto no pretende convertirse en una plataforma de inversión real ni reemplazar a un asesor financiero. Su valor está en combinar gestión financiera personal con un asistente inteligente de propósito educativo. Este asistente analiza patrones de gasto, clasifica el perfil financiero del usuario y ofrece explicaciones sencillas sobre ahorro, riesgo, interés compuesto y simulaciones de inversión hipotéticas.

Desde el punto de vista de diseño y arquitectura de software, el sistema permite aplicar varios conceptos trabajados en el curso: modelamiento orientado a objetos con UML, patrones de diseño GoF, atributos de calidad y estilos arquitectónicos. La propuesta se formula antes de escribir código, con el fin de justificar cada decisión técnica desde el problema del dominio y no desde la tecnología disponible.

2. Descripción del problema

Una parte importante de los estudiantes universitarios maneja ingresos limitados, variables o informales. Estos ingresos pueden provenir de mesadas, trabajos de medio tiempo, prácticas, emprendimientos pequeños o apoyo familiar. Aunque el monto no siempre es alto, la forma de administrarlo sí tiene impacto directo en su bienestar diario: transporte, alimentación, materiales académicos, entretenimiento y ahorro.

El problema no es únicamente registrar gastos. Muchas personas pueden anotar lo que compran durante algunos días, pero abandonan el proceso porque no reciben una interpretación útil de sus datos. Sin análisis, el registro se vuelve una tarea repetitiva y no una herramienta para decidir mejor.

También existe una brecha de educación financiera. Conceptos como presupuesto, fondo de emergencia, capacidad de ahorro, perfil de riesgo, diversificación e interés compuesto suelen aparecer tarde en la vida académica o laboral. Como resultado, algunos usuarios toman decisiones financieras guiados por tendencias, recomendaciones informales o promesas de rentabilidad poco claras.

FinWise AI propone atender este problema desde una solución gradual. Primero, permite organizar ingresos, gastos y metas. Luego, transforma esos datos en reportes comprensibles. Finalmente, usa un asistente inteligente para explicar hallazgos y simular escenarios hipotéticos de ahorro e inversión, sin convertir la aplicación en un intermediario financiero.

2.1 Pregunta orientadora del proyecto

¿Cómo diseñar un sistema de software que ayude a estudiantes universitarios a comprender y mejorar sus hábitos financieros mediante registro de datos, análisis inteligente, alertas y simulaciones educativas de inversión?

3. Objetivo del proyecto

Desarrollar un sistema de información que permita a estudiantes universitarios gestionar sus ingresos, gastos, presupuestos y metas de ahorro, con el fin de mejorar su control financiero personal y fortalecer su educación en inversión mediante análisis inteligente, simulaciones y recomendaciones educativas basadas en su perfil financiero.

3.1 Justificación del objetivo

- El objetivo se centra en el dominio financiero personal, no en una tecnología específica. Esto permite mantener clara la diferencia entre el problema que se quiere resolver y las herramientas que podrían usarse para implementarlo.
- El usuario principal está delimitado: estudiantes universitarios. Esta decisión ayuda a acotar el lenguaje, las funcionalidades y las métricas de calidad. Un sistema para estudiantes no tendría las mismas necesidades que una plataforma bancaria empresarial.
- El componente inteligente se entiende como apoyo educativo. Su propósito es profundizar el análisis, explicar los resultados y proponer simulaciones, no tomar decisiones por el usuario.

4. Alcance del sistema

4.1 Funcionalidades incluidas

- Registro e inicio de sesión de usuarios.
- Registro de ingresos y gastos.
- Clasificación de transacciones por categoría.
- Creación y seguimiento de presupuestos mensuales.
- Creación y seguimiento de metas de ahorro.
- Generación de reportes financieros básicos.
- Alertas por exceso de presupuesto o gastos inusuales.
- Clasificación del perfil financiero y de riesgo del usuario.
- Orientación educativa generada por el asistente inteligente.
- Simulaciones hipotéticas de inversión y ahorro.

4.2 Funcionalidades excluidas

- Ejecución real de inversiones o transferencias de dinero.
- Recomendaciones directas sobre compra o venta de activos específicos.
- Integración obligatoria con cuentas bancarias reales en la primera versión.
- Administración de portafolios de inversión reales.
- Gestión contable empresarial o tributaria.

5. Actores y módulos principales

5.1 Actores del sistema

Actor	Descripción
Usuario	Estudiante o joven adulto que registra sus datos financieros, consulta reportes y usa el asistente educativo.
Administrador	Persona encargada de gestionar categorías base, parámetros generales y contenido educativo inicial.
Asistente IA	Componente del sistema que analiza datos financieros y produce explicaciones, alertas y recomendaciones educativas.
Servicio externo financiero	Fuente futura o simulada de indicadores generales, como tasas de referencia o información educativa del mercado.

5.2 Módulos del sistema

Módulo	Responsabilidad principal
Usuarios	Gestionar registro, autenticación, perfil y preferencias del usuario.
Transacciones	Registrar ingresos y gastos, mantener historial y clasificar movimientos.
Categorías	Administrar categorías financieras predefinidas o personalizadas.
Presupuestos	Definir límites de gasto y controlar el consumo por periodo.
Metas de ahorro	Crear objetivos financieros y medir su avance.
Reportes	Generar resúmenes mensuales, por categoría, presupuesto y ahorro.
Alertas	Notificar excesos, gastos inusuales y metas en riesgo.
Asistente IA	Analizar hábitos y explicar recomendaciones de forma educativa.
Simulación	Proyectar escenarios hipotéticos de ahorro e inversión según variables ingresadas.

6. Requerimientos funcionales y reglas de negocio

6.1 Requerimientos funcionales

ID	Nombre	Descripción	Actor	Prioridad
RF-001	Registro de usuario	El sistema debe permitir que un usuario cree una cuenta proporcionando nombre, correo electrónico y contraseña.	Usuario	Alta
RF-002	Inicio de sesión	El sistema debe permitir que el usuario acceda a su cuenta mediante credenciales válidas.	Usuario	Alta
RF-003	Registro de transacciones	El sistema debe permitir registrar ingresos y gastos indicando monto, fecha, categoría y descripción.	Usuario	Alta
RF-004	Gestión de categorías	El sistema debe permitir clasificar las transacciones en categorías financieras predefinidas o personalizadas.	Usuario	Media
RF-005	Gestión de presupuestos	El sistema debe permitir crear presupuestos mensuales y asignar límites de gasto por categoría.	Usuario	Alta
RF-006	Gestión de metas de ahorro	El sistema debe permitir crear metas de ahorro con monto objetivo, fecha límite y seguimiento de progreso.	Usuario	Alta
RF-007	Generación de reportes	El sistema debe permitir generar reportes financieros sobre ingresos, gastos, presupuestos y metas de ahorro.	Usuario	Alta
RF-008	Alertas financieras	El sistema debe generar alertas cuando el usuario esté cerca de exceder un presupuesto, tenga gastos inusuales o incumpla una meta.	Sistema	Alta
RF-009	Asistente financiero inteligente	El sistema debe permitir que el usuario reciba orientación educativa basada en sus hábitos de gasto, capacidad de ahorro y perfil de riesgo.	Usuario	Alta

ID	Nombre	Descripción	Actor	Prioridad
RF-010	Simulación de inversión	El sistema debe permitir simular escenarios hipotéticos de inversión según monto, plazo, aporte mensual y perfil de riesgo.	Usuario	Media

6.2 Reglas de negocio

ID	Regla de negocio	RF relacionado
RN-001	Una transacción no puede registrarse si no tiene monto, fecha y categoría asociada.	RF-003
RN-002	El usuario no puede crear una meta de ahorro sin definir monto objetivo y fecha límite.	RF-006
RN-003	Si el gasto acumulado de una categoría supera el 90% del presupuesto asignado, el sistema debe generar una alerta preventiva.	RF-005, RF-008
RN-004	Si el gasto de una categoría supera el 100% del presupuesto, el sistema debe marcar la categoría como excedida.	RF-005, RF-008
RN-005	El asistente financiero no puede emitir instrucciones directas de compra o venta de activos específicos; solo puede entregar orientación educativa, simulaciones y recomendaciones generales.	RF-009, RF-010
RN-006	Antes de sugerir escenarios de inversión, el sistema debe evaluar si el usuario cuenta con capacidad de ahorro positiva.	RF-009, RF-010

7. Diagrama de clases UML

El modelo de clases se organiza alrededor del usuario y su información financiera. Las clases Usuario, CuentaFinanciera, Transaccion, Presupuesto, MetaAhorro y ReporteFinanciero representan el núcleo funcional del sistema. El componente de inteligencia se modela mediante AsistenteIA, PerfilFinanciero, RecomendacionEducativa y SimulacionInversion.

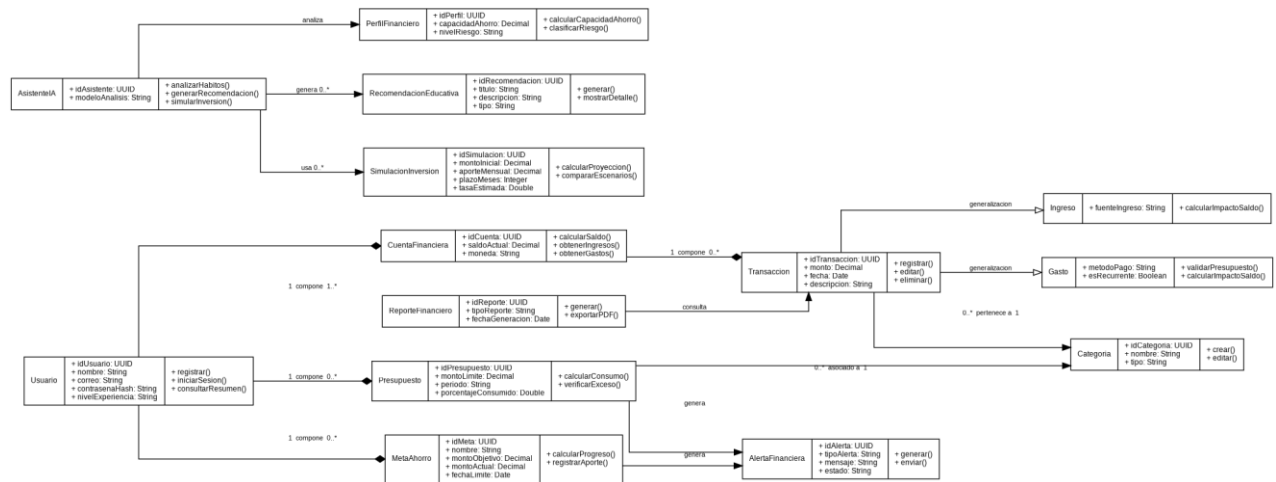


Figura 1. Diagrama de clases UML propuesto para FinWise AI.

7.1 Descripción de clases principales

Clase	Responsabilidad	Métodos relevantes
Usuario	Representa a la persona que administra sus finanzas dentro del sistema.	registrar(), iniciarSesion(), consultarResumen()
CuentaFinanciera	Agrupar el saldo y las transacciones registradas por el usuario.	calcularSaldo(), obtenerIngresos(), obtenerGastos()
Transaccion	Clase base para movimientos financieros.	registrar(), editar(), eliminar()
Ingreso	Especialización de Transaccion que aumenta el saldo disponible.	calcularImpactoSaldo()
Gasto	Especialización de Transaccion que disminuye el saldo y puede afectar presupuestos.	validarPresupuesto(), calcularImpactoSaldo()
Categoria	Permite clasificar ingresos y gastos por tipo.	crear(), editar()
Presupuesto	Define límites de gasto por categoría y periodo.	calcularConsumo(), verificarExceso()
MetaAhorro	Representa un objetivo financiero con monto y fecha límite.	calcularProgreso(), registrarAporte()
AlertaFinanciera	Notificación generada por eventos relevantes del sistema.	generar(), enviar()
AsistenteIA	Componente que analiza hábitos y genera orientación educativa.	analizarHabitos(), generarRecomendacion(), simularInversion()
PerfilFinanciero	Resume capacidad de ahorro y nivel de riesgo del usuario.	calcularCapacidadAhorro(), clasificarRiesgo()
SimulacionInversion	Calcula escenarios hipotéticos de inversión.	calcularProyeccion(), compararEscenarios()

7.2 Relaciones UML relevantes

Relación	Tipo	Justificación
Usuario - CuentaFinanciera	Composición	La cuenta financiera pertenece al usuario dentro del sistema.
CuentaFinanciera - Transaccion	Composición	Las transacciones existen asociadas a una cuenta.
Transaccion - Ingreso/Gasto	Herencia	Ingreso y Gasto son tipos específicos de transacción.
Transaccion - Categoria	Asociación	Cada transacción pertenece a una categoría.
Usuario - Presupuesto	Composición	Los presupuestos son definidos por cada usuario.
Usuario - MetaAhorro	Composición	Las metas de ahorro son personales y dependen del usuario.
Presupuesto - AlertaFinanciera	Dependencia	Un presupuesto puede generar alertas cuando se aproxima o supera el límite.
AsistenteIA - PerfilFinanciero	Asociación	El asistente analiza el perfil para generar orientación.
AsistenteIA - RecomendacionEducativa	Asociación	El asistente produce recomendaciones educativas.

8. Evaluación de patrones de diseño aplicables

Los patrones seleccionados responden a problemas específicos del dominio. No se incluyen como una lista decorativa, sino como mecanismos para mantener el sistema flexible, entendible y preparado para cambios en reportes, recomendaciones, alertas e integraciones futuras.

8.1 Strategy - Categoría: comportamiento

Problema del dominio: el sistema no debe analizar a todos los usuarios con el mismo criterio. Un usuario conservador, uno moderado y uno arriesgado requieren explicaciones y simulaciones diferentes.

Aplicación: se define una interfaz `EstrategiaRecomendacion` y varias implementaciones: `EstrategiaConservadora`, `EstrategiaModerada` y `EstrategiaArriesgada`. El `AsistenteIA` selecciona la estrategia según el `PerfilFinanciero`.

Consecuencias: el patrón facilita agregar nuevas estrategias sin modificar la lógica principal del asistente. Como desventaja, aumenta el número de clases y exige mantener nombres claros para evitar confusión.

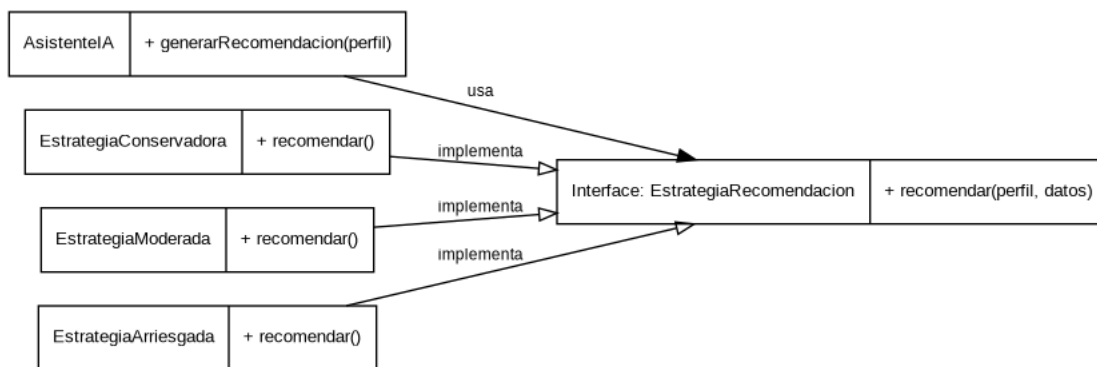


Figura 2. Aplicación del patrón Strategy al módulo de recomendaciones.

8.2 Observer - Categoría: comportamiento

Problema del dominio: cuando se registra un gasto, varios módulos necesitan reaccionar. El presupuesto debe actualizarse, las alertas pueden dispararse, los reportes deben quedar preparados y el asistente puede recalcular indicadores.

Aplicación: el servicio de transacciones publica el evento de registro o modificación. Los observadores `ServicioPresupuesto`, `ServicioAlertas`, `ServicioReportes` y `ServicioAnálisisIA` reaccionan sin que `TransaccionService` tenga que conocer sus detalles internos.

Consecuencias: se reduce el acoplamiento entre módulos. El costo es que el flujo completo puede ser menos evidente si no se documentan bien los eventos.

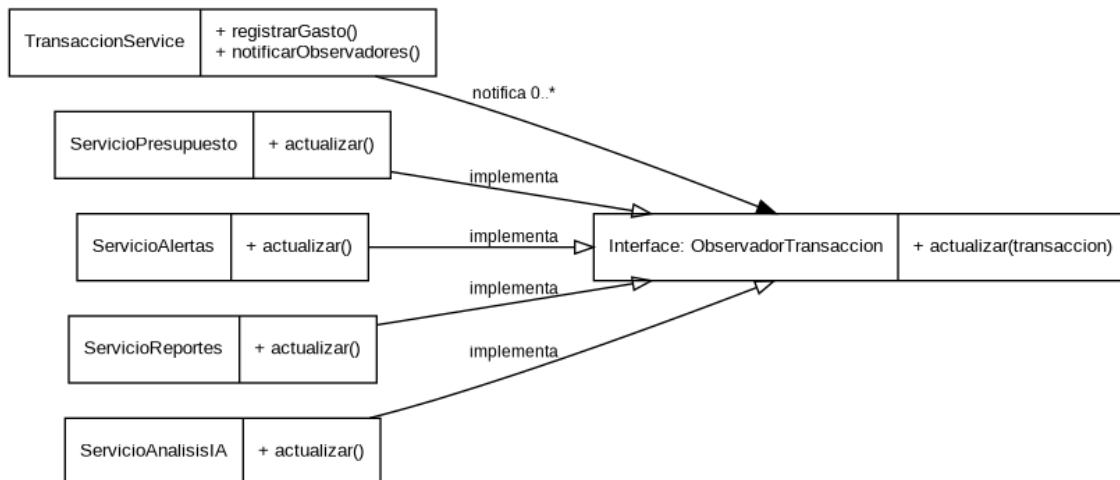


Figura 3. Aplicación del patrón Observer al registro de transacciones.

8.3 Factory Method - Categoría: creacional

Problema del dominio: el sistema debe generar reportes de diferentes tipos: mensual, por categoría, de ahorro, de presupuesto y de simulación. Crear cada reporte directamente desde la interfaz haría crecer el acoplamiento.

Aplicación: ReporteFactory centraliza la creación de reportes y devuelve una instancia concreta según el tipo solicitado. Todos los reportes comparten la abstracción ReporteFinanciero.

Consecuencias: se facilita incorporar nuevos reportes sin modificar las pantallas del usuario. La desventaja es que la fábrica debe mantenerse sincronizada con los tipos soportados.

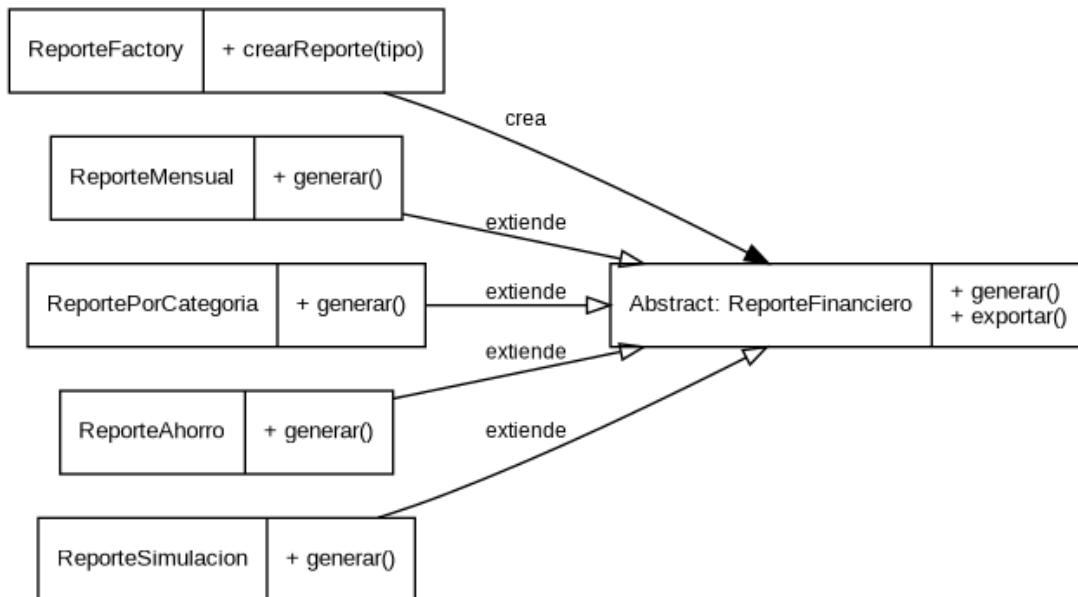


Figura 4. Aplicación del patrón Factory Method para la generación de reportes.

8.4 Facade - Categoría: estructural

Problema del dominio: el subsistema inteligente puede involucrar datos de transacciones, presupuestos, perfil financiero, simulaciones y recomendaciones. La interfaz no debería conocer toda esa complejidad.

Aplicación: AsistenteFinancieroFacade ofrece operaciones simples como generarDiagnostico(), obtenerRecomendaciones() y simularInversion(). Internamente coordina las clases AsistenteIA, PerfilFinanciero, SimulacionInversion y RecomendacionEducativa.

Consecuencias: simplifica el uso del módulo inteligente y protege a la interfaz de cambios internos. El riesgo es convertir la fachada en una clase demasiado grande si no se controla su responsabilidad.

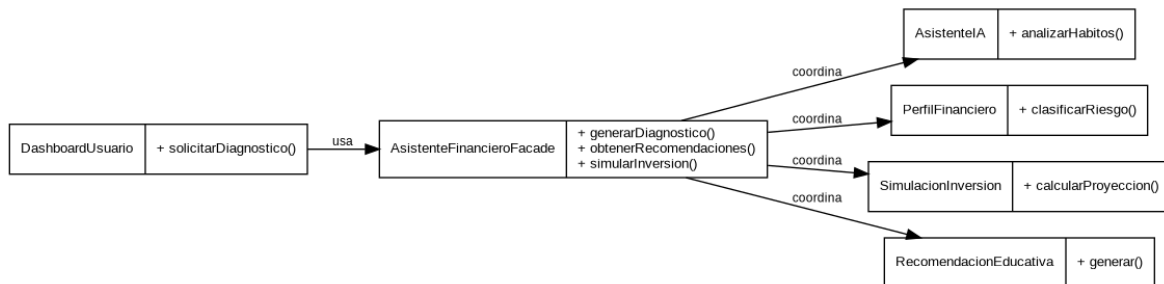


Figura 5. Aplicación del patrón Facade al subsistema inteligente.

8.5 Adapter - Categoría: estructural

Problema del dominio: en una evolución futura, el sistema podría consultar indicadores financieros generales desde APIs externas. Esas APIs no necesariamente tendrán el mismo formato que las clases internas del sistema.

Aplicación: se define la interfaz ProveedorDatosFinancieros. ApiIndicadoresAdapter convierte la respuesta de una API externa al formato que necesita MotorSimulacion.

Consecuencias: se protege el núcleo del sistema frente a cambios de proveedores externos. Como trade-off, se agrega una capa más de abstracción.

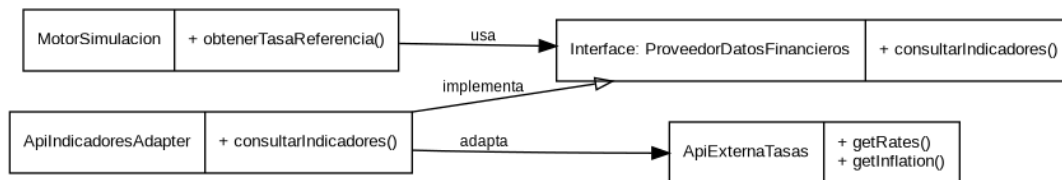


Figura 6. Aplicación del patrón Adapter para integraciones financieras futuras.

9. Atributos de calidad

Los atributos de calidad seleccionados son los que más afectan las decisiones de diseño del sistema. Aunque podrían considerarse otros, esta primera entrega prioriza cinco: seguridad, usabilidad, mantenibilidad, rendimiento y explicabilidad.

Atributo	Definición aplicada	Escenario	Métrica	Impacto en diseño
Security	Protección de datos personales y financieros del usuario.	Un usuario consulta su historial y solo él debe poder acceder a sus datos.	100% de contraseñas cifradas; rutas privadas con autenticación; acceso filtrado por propietario.	Autenticación, autorización, cifrado de contraseñas y separación de roles.
Usability	Facilidad para que usuarios no expertos comprendan sus finanzas.	Un estudiante registra un gasto y revisa si va bien frente al presupuesto.	Registro de gasto en menos de 1 minuto; reportes claros; textos educativos simples.	Dashboard, formularios breves, alertas visibles y explicaciones en lenguaje cotidiano.
Maintainability	Capacidad de modificar reglas, reportes y estrategias sin afectar todo el sistema.	El equipo agrega una nueva estrategia para usuarios con perfil muy conservador.	Agregar estrategia sin tocar AsistenteIA; módulos con responsabilidades separadas.	Monolito modular, Strategy, Factory Method y separación por paquetes.
Performance	Respuesta rápida en consultas y reportes frecuentes.	El usuario abre el resumen mensual con cientos de transacciones registradas.	Dashboard menor a 2 segundos; reporte mensual menor a 3 segundos con hasta 1.000 transacciones.	Consultas optimizadas, índices, agregados y posible caché de reportes.
Explainability	Capacidad de explicar por qué se genera una alerta o recomendación.	La IA indica bajo ahorro y debe justificarlo con datos del usuario.	Toda recomendación debe incluir una razón basada en indicadores financieros.	Recomendaciones trazables, variables visibles y simulaciones marcadas como hipotéticas.

9.1 Atributo dominante

El atributo dominante del sistema es Security. FinWise AI maneja información sensible: ingresos, gastos, hábitos de consumo, capacidad de ahorro, metas personales y perfil de riesgo. Si el usuario no confía en la protección de sus datos, el resto de funcionalidades pierde valor.

La seguridad también condiciona la arquitectura. Obliga a separar responsabilidades, validar acceso por usuario, cifrar credenciales, evitar exposición innecesaria de datos y controlar qué información puede usar el asistente inteligente.

10. Estilo arquitectónico y evolución

10.1 Estilo inicial propuesto: monolito modular

Para la primera versión, se propone un monolito modular. Este estilo mantiene un único despliegue, pero organiza internamente el sistema en módulos con responsabilidades claras: usuarios, transacciones, presupuestos, metas, reportes, alertas, asistente IA y simulación.

La decisión es realista para un equipo académico de hasta cinco estudiantes. Iniciar con microservicios aumentaría la complejidad operacional sin que el dominio lo exija todavía. En cambio, un monolito modular permite desarrollar, probar y depurar de forma más sencilla, sin renunciar a una separación interna ordenada.

El estilo favorece mantenibilidad y testabilidad, siempre que se respeten límites entre módulos. Puede dificultar la escalabilidad independiente, porque todos los módulos se despliegan juntos. Esa limitación se acepta en la primera etapa y se aborda en la ruta de evolución.

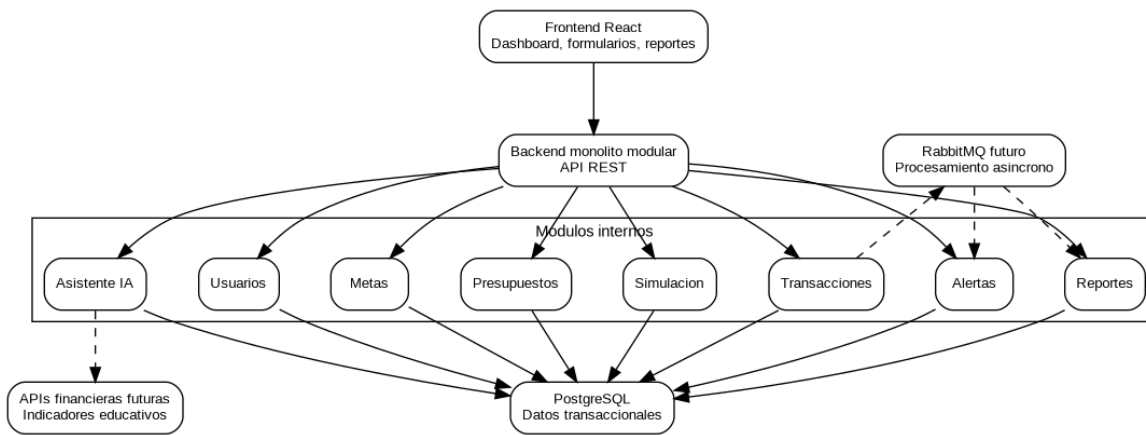


Figura 7. Vista inicial de arquitectura propuesta como monolito modular.

10.2 Mapeo de módulos al estilo arquitectónico

Módulo	Responsabilidad arquitectónica
Usuarios	Autenticación, perfil y preferencias.
Transacciones	Registro y consulta de ingresos y gastos.
Presupuestos	Control de límites por categoría y periodo.
Metas	Seguimiento de objetivos de ahorro.
Reportes	Generación de resúmenes e indicadores.
Alertas	Notificaciones internas por eventos financieros.
Asistente IA	Análisis de hábitos, explicación y orientación educativa.
Simulación	Cálculo de escenarios hipotéticos de ahorro e inversión.

10.3 Ruta de evolución arquitectónica

Hito	Gatillo	Estilo	Cambio estructural	Tácticas y patrones
Hito 1 - Inicio	Equipo pequeño, dominio acotado y plazo académico.	Monolito modular.	Módulos internos organizados por responsabilidad.	Separación por capas, Strategy, Factory Method y Facade.
Hito 2 - Crecimiento	Los reportes y análisis IA empiezan a consumir más tiempo de procesamiento.	Monolito modular con procesamiento asíncrono.	Reportes, alertas y análisis se ejecutan mediante eventos o cola de mensajes.	Observer, colas, caché y tareas en segundo plano.
Hito 3 - Escala	El sistema llega a muchos usuarios y algunos módulos requieren escalar de forma independiente.	Arquitectura orientada a servicios o microservicios parciales.	Se extraen servicios de IA, reportes y notificaciones.	API Gateway, Adapter para APIs externas, mensajería y despliegues independientes.

11. Stack tecnológico propuesto

La selección tecnológica se plantea como una propuesta coherente con el monolito modular. No se escogen herramientas por moda, sino por su relación con los atributos de calidad y con la capacidad realista del equipo para implementarlas.

Capa	Decisión tecnológica	Justificación
Frontend / UI	React	Permite construir dashboards, formularios y visualizaciones interactivas para reportes y simulaciones.
Backend / lógica de negocio	Spring Boot	Facilita organizar un monolito modular, aplicar principios orientados a objetos y exponer servicios REST.
Persistencia	PostgreSQL	Los datos son estructurados y relacionales: usuarios, cuentas, transacciones, presupuestos y metas.
IA / análisis	Python o servicio interno de análisis	Permite prototipar clasificación de perfil, reglas de recomendación y simulaciones educativas.
Mensajería futura	RabbitMQ	Justificada cuando alertas, reportes o análisis se procesen de forma asíncrona. No es obligatoria en la primera versión.
Infraestructura	Docker	Ayuda a ejecutar backend, base de datos y servicios auxiliares de forma reproducible.
CI/CD	GitHub Actions	Permite ejecutar validaciones básicas, pruebas y revisión de calidad del proyecto.
Control de versiones	Git + GitHub	Soporta trabajo colaborativo, historial de cambios y entrega del repositorio académico.

11.1 Alternativa si el equipo no usa Spring Boot

Si el equipo se siente más cómodo con JavaScript, el backend podría implementarse con Node.js y Express. Sin embargo, para esta entrega se recomienda Spring Boot porque facilita explicar clases, interfaces, patrones GoF y una organización modular más cercana al enfoque de diseño orientado a objetos trabajado en clase.

12. Repositorio y estructura del proyecto

El repositorio de GitHub será el espacio oficial del proyecto. En esta primera entrega no es indispensable tener toda la aplicación implementada, pero sí debe existir una estructura inicial clara, un README completo y los artefactos principales del análisis.

```
finwise-ai/
|
|-- README.md
|-- docs/
|   |-- documento-entrega-1.pdf
|   |-- diagramas/
|       |-- diagrama-clases.png
|       |-- diagrama-patron-strategy.png
|       |-- diagrama-patron-observer.png
|       |-- diagrama-arquitectura.png
|   |-- referencias/
|       |-- referencias-apa.md
|
|-- backend/
|   |-- README.md
|   |-- src/
|       |-- usuarios/
|       |-- transacciones/
|       |-- presupuestos/
```

```

|         |-- metas/
|         |-- reportes/
|         |-- alertas/
|         |-- asistente_ia/
|
|-- frontend/
|   |-- README.md
|   |-- src/
|     |-- components/
|     |-- pages/
|     |-- services/
|
|-- ai-service/
|   |-- README.md
|   |-- analysis/
|   |-- models/
|
|-- database/
|   |-- modelo-datos.md
|   |-- schema.sql
|
|-- docker/
|   |-- docker-compose.yml

```

12.1 Contenido mínimo del README.md

Elemento	Contenido esperado
Nombre del proyecto	FinWise AI.
Descripción	Sistema inteligente de gestión financiera personal y educación en inversión.
Objetivo	Resumen del objetivo general del sistema.
Integrantes	Nombre completo y código de cada estudiante.
Módulos	Usuarios, transacciones, presupuestos, metas, reportes, alertas, IA y simulación.
Stack	React, Spring Boot, PostgreSQL, Python, Docker y GitHub.
Advertencia de alcance	El asistente es educativo y no realiza asesoría financiera directa ni inversiones reales.

13. Riesgos, supuestos y decisiones relevantes

13.1 Supuestos del proyecto

- El usuario ingresará manualmente sus ingresos y gastos en la primera versión.
- El sistema trabajará con una sola moneda principal configurada por el usuario.
- Las simulaciones de inversión usarán tasas hipotéticas o de referencia, no promesas de rentabilidad.
- La clasificación de perfil financiero se basará en reglas y datos declarados por el usuario en la primera versión.
- La integración con APIs financieras se considera una evolución futura, no una obligación del primer prototipo.

13.2 Riesgos identificados

Riesgo	Impacto	Mitigación
El usuario interpreta la orientación educativa como recomendación de inversión directa.	Alto	Mensajes claros de alcance, reglas de negocio y explicaciones de que las simulaciones son hipotéticas.

Riesgo	Impacto	Mitigación
El asistente genera respuestas poco claras o difíciles de justificar.	Medio	Registrar indicadores usados y acompañar cada recomendación con una explicación breve.
El modelo de clases crece demasiado y pierde legibilidad.	Medio	Separar por módulos y mantener solo clases relevantes en el diagrama principal.
El equipo intenta implementar demasiadas funcionalidades en poco tiempo.	Alto	Priorizar registro, presupuestos, reportes y una IA educativa básica.
Falta de protección sobre datos financieros del usuario.	Alto	Autenticación, autorización por usuario y manejo cuidadoso de credenciales.

14. Conclusiones

FinWise AI es una propuesta viable para el proyecto final porque tiene un dominio claro, usuarios definidos, reglas de negocio concretas y suficientes elementos para demostrar diseño orientado a objetos. A diferencia de un gestor simple de gastos, el componente inteligente permite profundizar en análisis, simulaciones y explicaciones educativas.

El sistema también es adecuado para aplicar patrones de diseño. Strategy permite variar la lógica de recomendación, Observer desacopla las reacciones ante nuevas transacciones, Factory Method organiza la creación de reportes, Facade simplifica el subsistema inteligente y Adapter prepara la integración futura con servicios externos.

La arquitectura inicial recomendada es un monolito modular. Esta decisión es coherente con el tamaño del equipo y el contexto académico, pero al mismo tiempo permite proponer una evolución razonada hacia procesamiento asíncrono o servicios independientes si el volumen de usuarios y datos aumenta.

En síntesis, el proyecto no solo plantea una aplicación útil, sino una base arquitectónica justificable. Cada sección conecta el problema financiero personal con decisiones de diseño, calidad, arquitectura y tecnología.

15. Referencias

- Bass, L., Clements, P., & Kazman, R. (2021). Software architecture in practice (4th ed.). Addison-Wesley.
- Ford, N., & Richards, M. (2020). Fundamentals of software architecture: An engineering approach. O'Reilly Media.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.
- Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley.
- Martin, R. C. (2017). Clean architecture: A craftsman's guide to software structure and design. Prentice Hall.
- International Organization for Standardization. (2011). ISO/IEC 25010:2011 Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - System and software quality models.